

700106 / 700120 Lab Book

Vulkan Exercises

Exercise 1: PREPARING FOR LIGHTING

Question: Update the application to support vertex normals and a new UBO for lighting data

Solution:

- The descriptor set layout remains binding 0 and VK_SHADER_STAGE_VERTEX_BIT, now used for model/view/proj plus lightPos and eyePos.
- After editing the GLSL, recompile to SPIR-V so the app can load shaders/vert.spv and shaders/frag.spv. Example:
 - glslc -fshader-stage=vert SHADERS/shader.vert -o shaders/vert.spv
 - glslc -fshader-stage=frag SHADERS/shader.frag -o shaders/frag.spv

What changed:

- Added normal to Vertex and third vertex attribute at location 2.
- Added lightPos and eyePos to UniformBufferObject and populated them in updateUniformBuffer.
- Replaced quad with a cube using per-face normals (no vertex sharing across faces).
- Implemented per-vertex lighting in the vertex shader; fragment shader outputs interpolated lit color.

```
struct UniformBufferObject {  
    alignas(16) glm::mat4 model;  
    alignas(16) glm::mat4 view;  
    alignas(16) glm::mat4 proj;  
    alignas(16) glm::vec3 lightPos;  
    alignas(16) glm::vec3 eyePos;  
};
```

```
1028
1029 // New lighting data
1030 ubo.eyePos = glm::vec3(2.0f, 2.0f, 2.0f);
1031 ubo.lightPos = glm::vec3(1.5f * std::cos(time), 1.0f, 1.5f * std::sin(time));
1032
```

```
shader.frag  × Lab_Tutorial_Template.cpp  shader.vert  ×
1  #version 450
2
3  layout(location = 0) in vec3 inPos;
4  layout(location = 1) in vec3 inColor;
5  layout(location = 2) in vec3 inNormal;
6
7  layout(location = 0) out vec3 fragColor;
8
9  layout(binding = 0) uniform UniformBufferObject {
10     mat4 model;
11     mat4 view;
12     mat4 proj;
13     vec3 lightPos; // std140 packs this to 16B
14     vec3 eyePos;   // std140 packs this to 16B
15 } ubo;
16
17 void main() {
18     vec4 worldPos = ubo.model * vec4(inPos, 1.0);
19
20     // Transform normal to world space
21     mat3 normalMatrix = transpose(inverse(mat3(ubo.model)));
22     vec3 N = normalize(normalMatrix * inNormal);
23
24     vec3 L = normalize(ubo.lightPos - worldPos.xyz);
25     vec3 V = normalize(ubo.eyePos - worldPos.xyz);
26
27     float ambient = 0.15;
28     float diffuse = max(dot(N, L), 0.0);
29
30     // Optional simple specular
31     vec3 R = reflect(-L, N);
32     float spec = pow(max(dot(R, V), 0.0), 16.0) * 0.25;
33
34     fragColor = inColor * (ambient + diffuse) + vec3(spec);
35
36     gl_Position = ubo.proj * ubo.view * worldPos;
37 }
```

Exercise 2: PER-VERTEX DIFFUSE LIGHTING

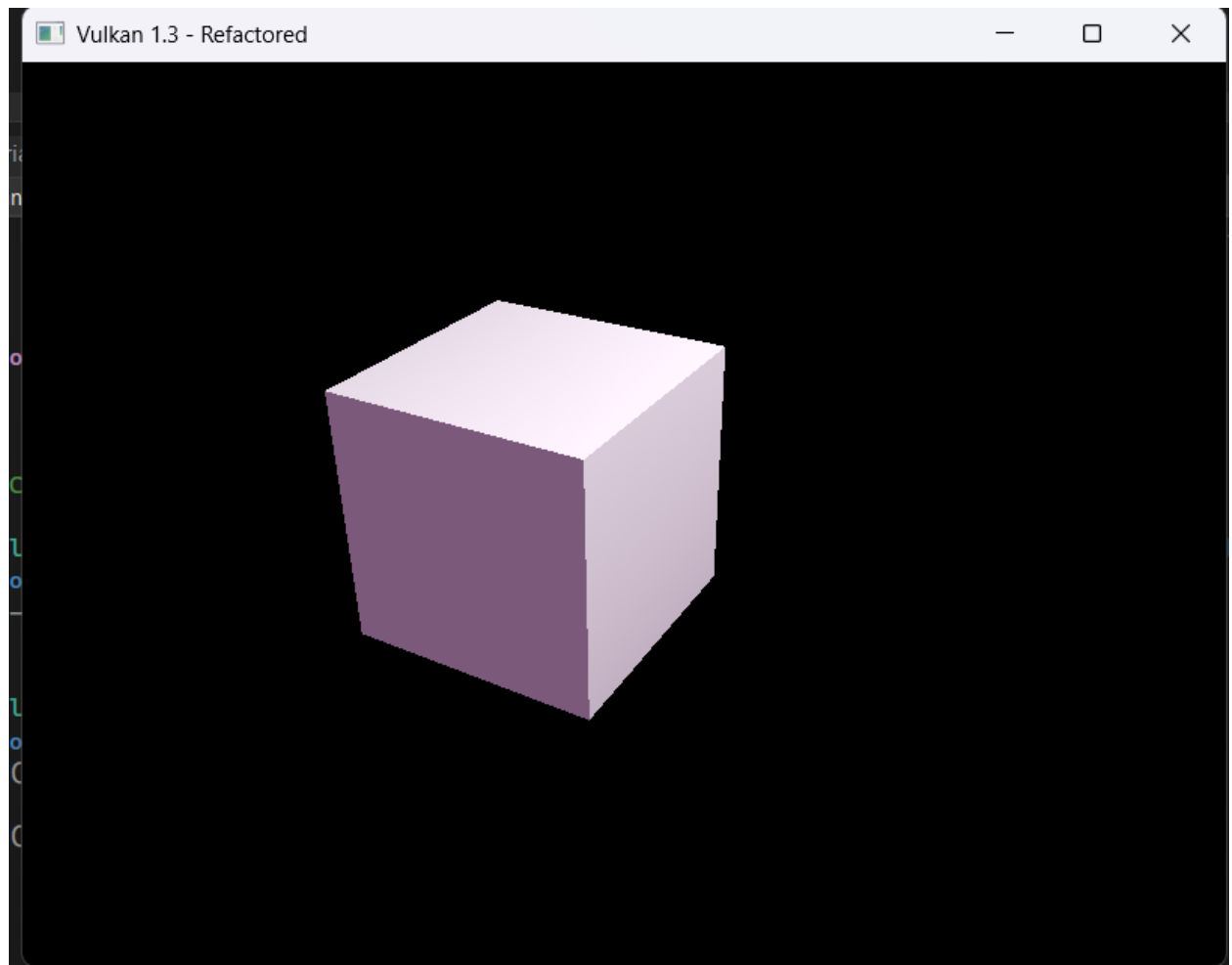
Question: Implement a basic ambient and diffuse lighting model directly in the vertex shader.

Reflections:

- Getting lighting right requires consistent spaces: compute lighting in world space if you build worldPos with model and transform normals with the normal matrix `transpose(inverse(mat3(model)))`.
- Shader interfaces must match exactly: location 0 out in the vertex must match location 0 in in the fragment; wrong names or locations won't compile/link.
- UBO layout matters: using vec3 fields in std140 requires 16-byte alignment (your C++ UBO uses `alignas(16)`, which prevents subtle layout bugs).
- Per-vertex lighting is fast but can look faceted on large triangles; finer meshes or per-fragment lighting smooths results at higher cost.

Questions:

- In Vulkan practice, when applying non-uniform scaling to the model matrix, is `transpose(inverse(mat3(model)))`



```

1      #version 450
2
3      layout(binding = 0) uniform UniformBufferObject {
4          mat4 model;
5          mat4 view;
6          mat4 proj;
7          vec3 lightPos;
8          vec3 eyePos;
9      } ubo;
10
11     layout(location = 0) in vec3 inPosition;
12     layout(location = 1) in vec3 inColor;
13     layout(location = 2) in vec3 inNormal;
14
15     layout(location = 0) out vec3 fragColor;
16
17     void main() {
18         // Transform position and normal to world space
19         vec3 worldPos = (ubo.model * vec4(inPosition, 1.0)).xyz;
20         mat3 normalMatrix = transpose(inverse(mat3(ubo.model)));
21         vec3 worldNormal = normalize(normalMatrix * inNormal);
22
23         // Light properties
24         vec3 lightColor = vec3(1.0, 1.0, 1.0);
25
26         // Ambient + diffuse
27         vec3 ambientMaterial = vec3(0.2, 0.1, 0.2);
28         vec3 L = normalize(ubo.lightPos - worldPos);
29         float diff = max(dot(worldNormal, L), 0.0);
30
31         // Treat vertex color as diffuse material
32         vec3 ambient = ambientMaterial * lightColor;
33         vec3 diffuse = diff * inColor * lightColor;
34
35         // Final color to fragment shader
36         fragColor = ambient + diffuse;
37
38         // Position must be a vec4
39         gl_Position = ubo.proj * ubo.view * vec4(worldPos, 1.0);
40     }

```

Exercise 3: PER-FRAGMENT DIFFUSE LIGHTING

Question: Improve visual quality by moving the lighting logic to the fragment shader, allowing for smoother, more accurate calculations.

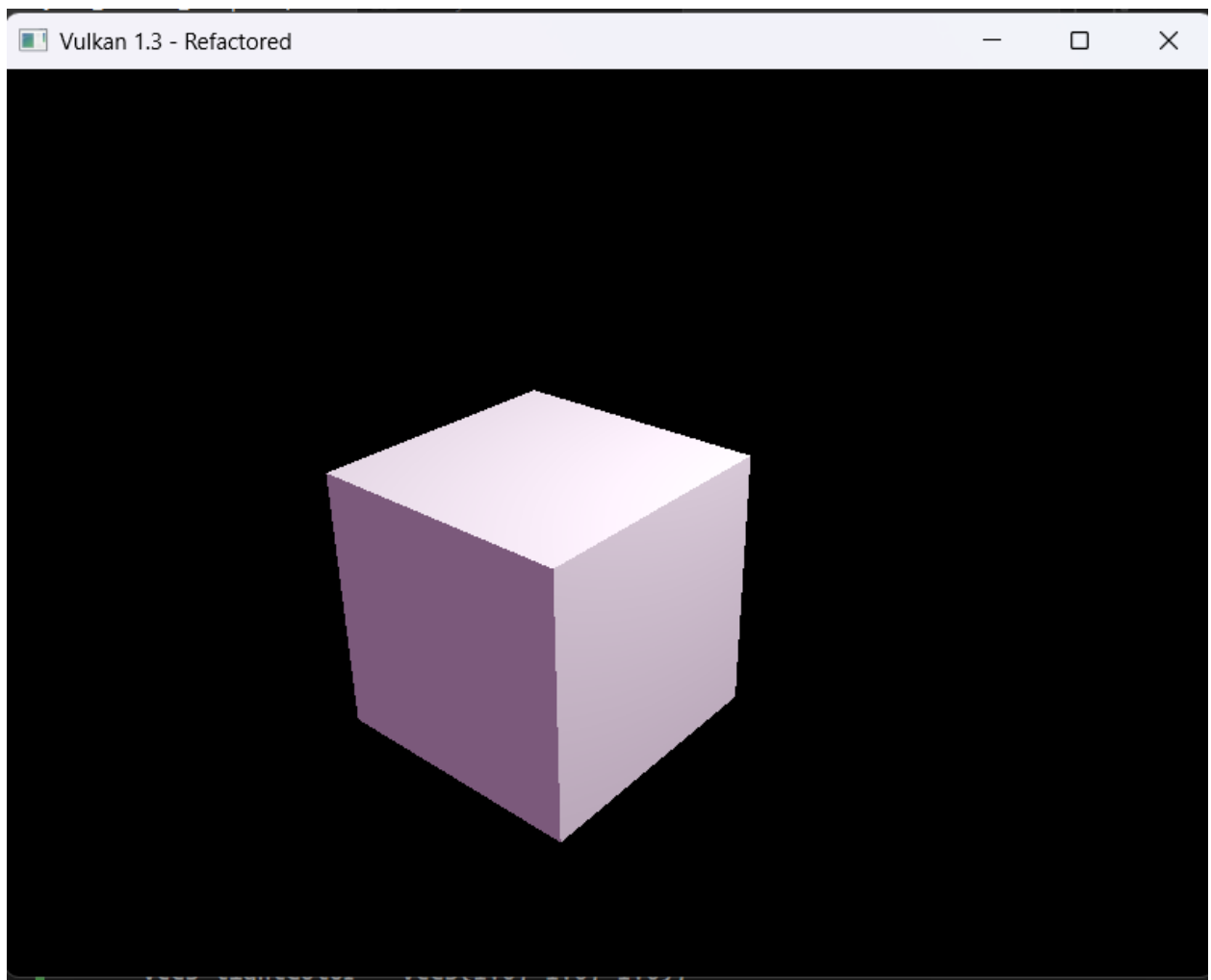
Solution: Switched to per-fragment diffuse lighting. The vertex shader now only computes and passes world-space position and normal, plus the vertex color, to the fragment shader. The fragment shader receives those interpolated values and performs the ambient + diffuse calculation. Because the UBO is read in the fragment stage, the descriptor set layout in C++ must expose binding 0 to both the vertex and fragment stages. After making these edits, recompile your shaders to SPIR-V and rebuild.

Reflection: Moving lighting to the fragment shader immediately improves visual smoothness because the lighting is evaluated per-pixel using interpolated attributes rather than per-vertex with barycentric interpolation of the final color. The key to correct results is staying consistent in spaces: positions and normals are converted into world space, normals use the normal matrix $\text{transpose}(\text{inverse}(\text{mat3}(\text{model})))$ to remain accurate under non-uniform scaling, and the light direction is computed from these world-space values. Ensuring the UBO layout matches between C++ and GLSL (including 16-byte alignment for vec3 fields) prevents subtle artefacts, and updating the descriptor stage flags is required so the fragment shader can access the same UBO binding.

Question: In practice with Vulkan and modern GPUs, do you recommend performing normal transformation with the full normal matrix in the shader for every fragment, or precomputing a simplified or orthonormalized basis on the CPU when non-uniform scaling is present, considering the trade-offs between accuracy, numerical stability, and performance for real-time rendering?

```
shader.frag  Lab_Tutorial_Template.cpp  shader.vert ↗ ✕
1      #version 450
2
3      layout(binding = 0) uniform UniformBufferObject {
4          mat4 model;
5          mat4 view;
6          mat4 proj;
7          vec3 lightPos;
8          vec3 eyePos;
9      } ubo;
10
11     layout(location = 0) in vec3 inPosition;
12     layout(location = 1) in vec3 inColor;
13     layout(location = 2) in vec3 inNormal;
14
15     layout(location = 0) out vec3 fragColor;
16     layout(location = 1) out vec3 fragWorldPos;
17     layout(location = 2) out vec3 fragWorldNormal;
18
19     void main() {
20         // World-space position and normal
21         vec3 worldPos = (ubo.model * vec4(inPosition, 1.0)).xyz;
22         mat3 normalMatrix = transpose(inverse(mat3(ubo.model)));
23         vec3 worldNormal = normalize(normalMatrix * inNormal);
24
25         // Pass to fragment
26         fragColor      = inColor;
27         fragWorldPos    = worldPos;
28         fragWorldNormal = worldNormal;
29
30         gl_Position = ubo.proj * ubo.view * vec4(worldPos, 1.0);
31     }
```

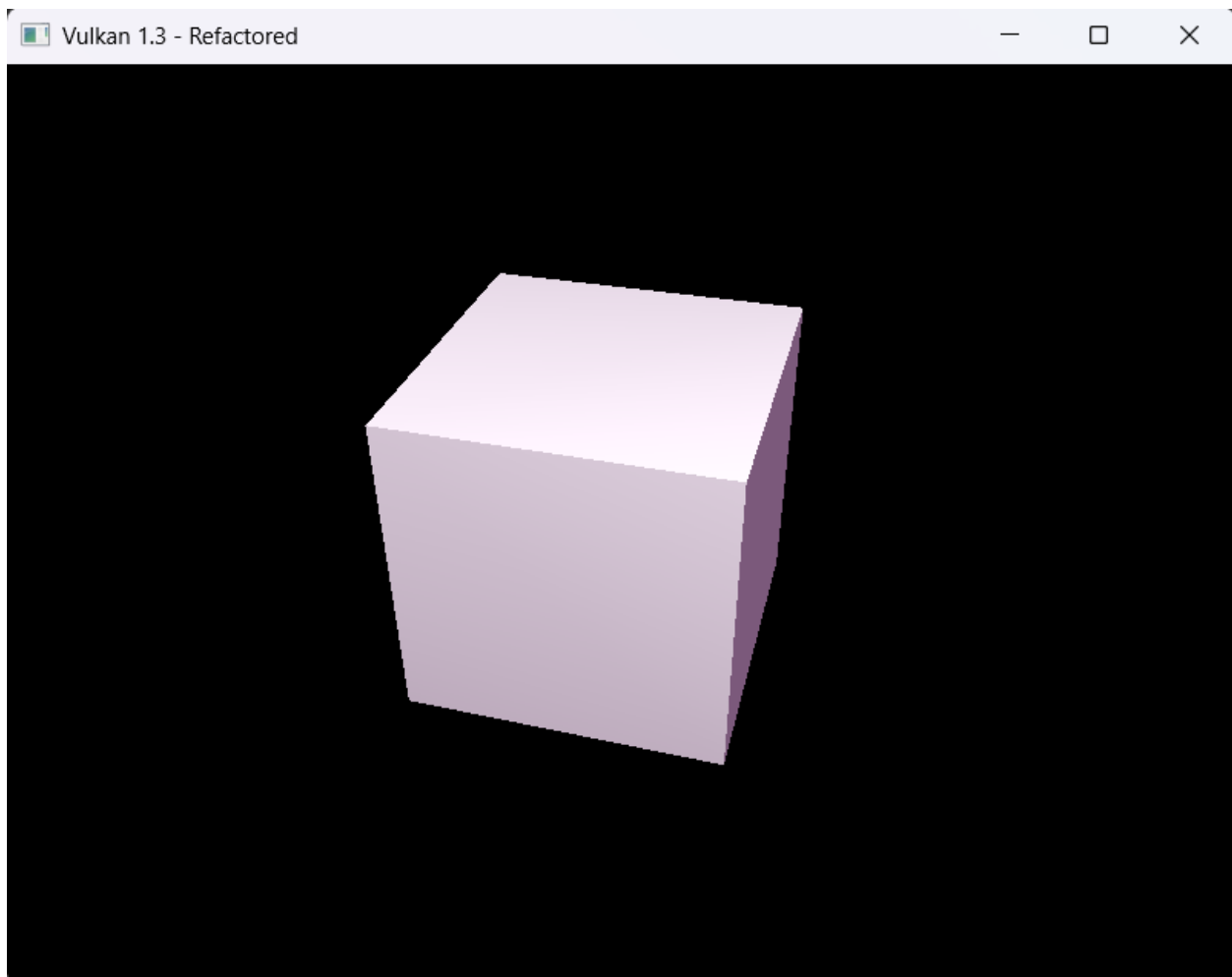
```
shader.frag ↗ ✕  Lab_Tutorial_Template.cpp  shader.vert
1      #version 450
2
3      layout(binding = 0) uniform UniformBufferObject {
4          mat4 model;
5          mat4 view;
6          mat4 proj;
7          vec3 lightPos;
8          vec3 eyePos;
9      } ubo;
10
11     layout(location = 0) in vec3 fragColor;
12     layout(location = 1) in vec3 fragWorldPos;
13     layout(location = 2) in vec3 fragWorldNormal;
14
15     layout(location = 0) out vec4 outColor;
16
17     void main() {
18         vec3 lightColor = vec3(1.0);
19         vec3 ambientMaterial = vec3(0.2, 0.1, 0.2);
20
21         vec3 N = normalize(fragWorldNormal);
22         vec3 L = normalize(ubo.lightPos - fragWorldPos);
23         float diff = max(dot(N, L), 0.0);
24
25         vec3 ambient = ambientMaterial * lightColor;
26         vec3 diffuse = diff * fragColor * lightColor;
27
28         outColor = vec4(ambient + diffuse, 1.0);
29     }
```



Exercise 4: ADDING PER-VERTEX SPECULAR LIGHTING

Question: Complete the reflection model implemented in Exercise 2 by adding specular highlights to the vertex shader.

Reflection: When targeting real-time pipelines, would you generally recommend Blinn-Phong (using a half-vector) rather than classic Phong reflection for the specular term in a per-vertex setup, and would evaluating in view space offer any practical benefits for numerical stability or performance compared to doing all specular math in world space as shown here?



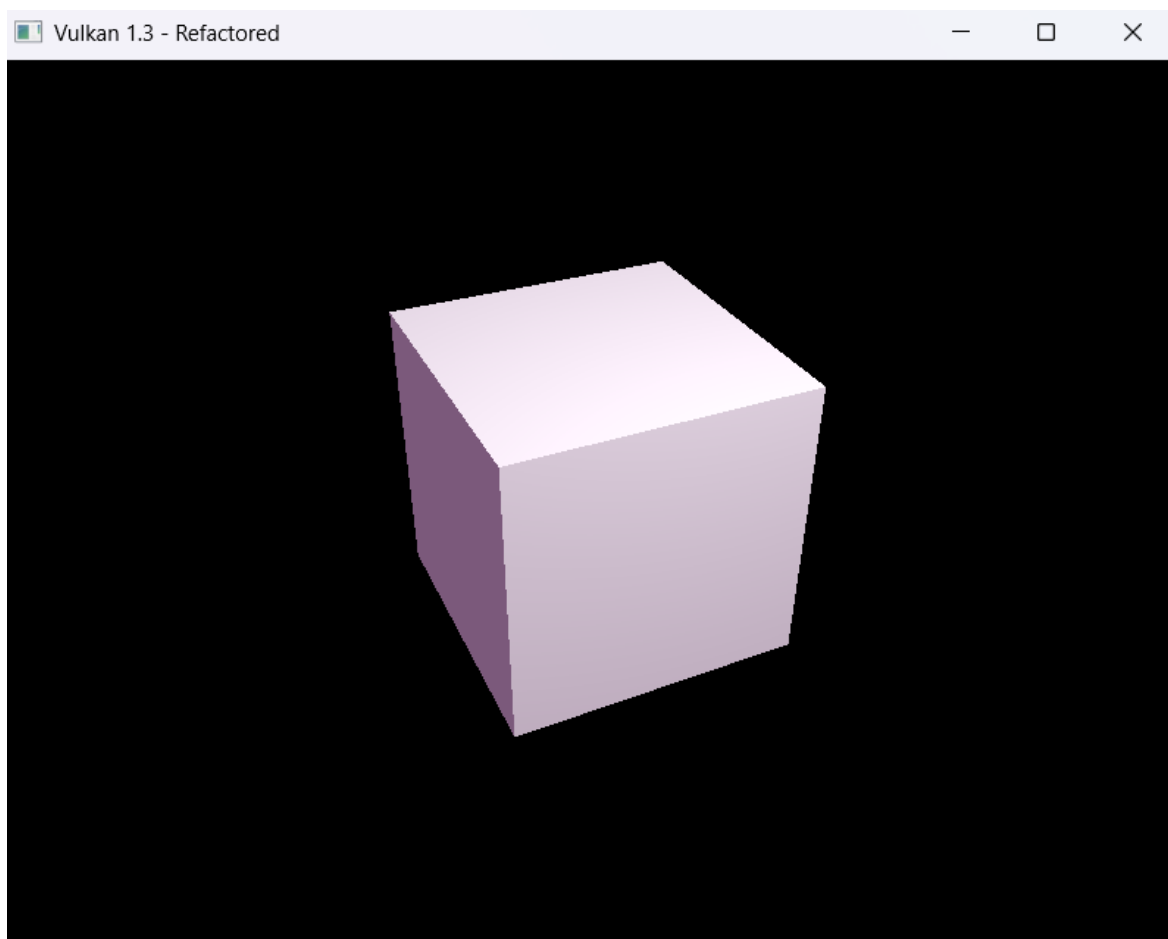
```
1  #version 450
2
3  layout(binding = 0) uniform UniformBufferObject {
4      mat4 model;
5      mat4 view;
6      mat4 proj;
7      vec3 lightPos;
8      vec3 eyePos;
9  } ubo;
10
11  layout(location = 0) in vec3 inPosition;
12  layout(location = 1) in vec3 inColor;
13  layout(location = 2) in vec3 inNormal;
14
15  layout(location = 0) out vec3 fragColor;
16
17  void main() {
18      // World-space position and normal
19      vec3 worldPos = (ubo.model * vec4(inPosition, 1.0)).xyz;
20      mat3 normalMatrix = transpose(inverse(mat3(ubo.model)));
21      vec3 norm = normalize(normalMatrix * inNormal);
22
23      // Lighting parameters
24      vec3 lightColor = vec3(1.0);
25      vec3 ambientMaterial = vec3(0.2, 0.1, 0.2);
26
27      // Diffuse (Lambert)
28      vec3 lightDir = normalize(ubo.lightPos - worldPos);
29      float diff = max(dot(norm, lightDir), 0.0);
30      vec3 diffuse = diff * inColor * lightColor;
31
32      // Specular (Phong)
33      vec3 viewDir = normalize(ubo.eyePos - worldPos);
34      vec3 reflectDir = normalize(reflect(-lightDir, norm));
35      float shininess = 32.0;
36      float spec = pow(max(dot(reflectDir, viewDir), 0.0), shininess);
37      vec3 specMaterial = vec3(1.0);
38      vec3 specular = specMaterial * lightColor * spec;
39
40      // Final vertex color (ambient + diffuse + specular)
41      vec3 ambient = ambientMaterial * lightColor;
42      fragColor = ambient + diffuse + specular;
43
44      gl_Position = ubo.proj * ubo.view * vec4(worldPos, 1.0);
45  }
```

Exercise 5: ADDING PER-FRAGMENT SPECULAR LIGHTING

Question: Complete the reflection model implemented in Exercise 3 by adding specular highlights to the fragment shader.

Solution: I extended the fragment shader to compute the Phong specular term per pixel. This uses the interpolated world-space normal and position along with the camera position from the UBO. Also, ensure your mesh actually has valid normals; your current **Cube_vertices** lack normals and will default to zeros, which flattens both diffuse and specular.

Reflection: Moving specular to the fragment stage produces sharper and more accurate highlights because the non-linear specular term is evaluated per pixel instead of being linearly interpolated from vertices. However, this improvement is only visible if your mesh supplies correct normals and your UBO values reflect your camera and light configuration; zero or incorrect normals will smear or eliminate highlights, and a mismatched eyePos and view matrix will misalign reflections.



```

shader.frag  × Lab_Tutorial_Template.cpp  shader.vert
1      #version 450
2
3      layout(binding = 0) uniform UniformBufferObject {
4          mat4 model;
5          mat4 view;
6          mat4 proj;
7          vec3 lightPos;
8          vec3 eyePos;
9      } ubo;
10
11     layout(location = 0) in vec3 fragColor;          // diffuse material color (from vertex)
12     layout(location = 1) in vec3 fragWorldPos;       // world-space position
13     layout(location = 2) in vec3 fragWorldNormal;    // world-space normal
14
15     layout(location = 0) out vec4 outColor;
16
17     void main() {
18         vec3 lightColor = vec3(1.0);
19
20         // Ambient
21         vec3 ambientMaterial = vec3(0.2, 0.1, 0.2);
22         vec3 ambient = ambientMaterial * lightColor;
23
24         // Diffuse
25         vec3 N = normalize(fragWorldNormal);
26         vec3 L = normalize(ubo.lightPos - fragWorldPos);
27         float diff = max(dot(N, L), 0.0);
28         vec3 diffuse = diff * fragColor * lightColor;
29
30         // Specular (Phong)
31         vec3 V = normalize(ubo.eyePos - fragWorldPos);
32         vec3 R = reflect(-L, N);
33         float shininess = 64.0;                      // increase for tighter highlights
34         float spec = pow(max(dot(R, V), 0.0), shininess);
35         vec3 specMaterial = vec3(1.0);              // white specular
36         vec3 specular = specMaterial * lightColor * spec;
37
38         outColor = vec4(ambient + diffuse + specular, 1.0);
39     }

```

Exercise 6: MULTIPLE LIGHTS AND MATERIALS

Question: Draw three cube objects with three different surface material properties. Illuminate these cubes with two light sources: one is a static white light; the other is a red light rotating dynamically by the y-axis.

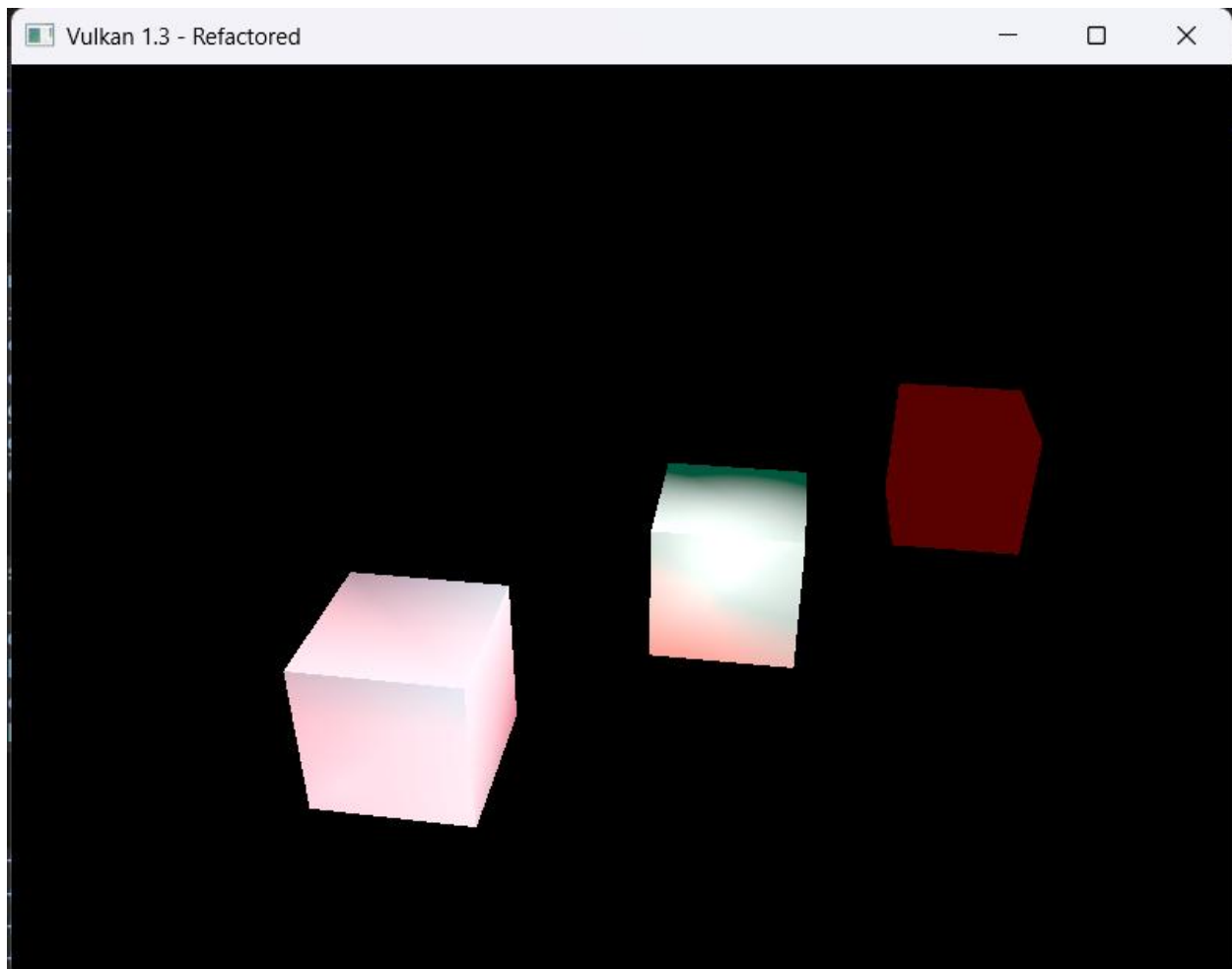
Solution: I extended the fragment shader to compute the Phong specular term per pixel. This uses the interpolated world-space normal and position along with the camera position from the UBO. Also, ensure your mesh actually has valid normals; your current **Cube_vertices** lack normals and will default to zeros, which flattens both diffuse and specular.

Reflection: This exercise introduced multiple light sources and material properties, showing how lighting equations are additive and modular. By separating light and material data, it becomes easy to extend scenes for realism. Animating a light by rotation adds dynamism and demonstrates how specular highlights react to light movement in real time.

Questions:

1. What's the computational cost of supporting multiple lights in fragment shaders, and how do engines typically optimize this (e.g., deferred rendering)?
2. Is it more efficient to handle material variation via UBO updates per draw, or to use descriptor sets for each material?
3. How would you extend this system to include attenuation (distance-based light falloff)?

```
1123 vkCmdEndRendering(commandBuffer);
1124
1125
1126 VkImageMemoryBarrier2 imageBarrierToPresent{};
1127 imageBarrierToPresent.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2;
1128 imageBarrierToPresent.srcStageMask = VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT;
1129 imageBarrierToPresent.srcAccessMask = VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT;
1130 imageBarrierToPresent.dstStageMask = VK_PIPELINE_STAGE_2_BOTTOM_OF_PIPE_BIT;
1131 imageBarrierToPresent.oldLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
1132 imageBarrierToPresent.newLayout = VK_IMAGE_LAYOUT_PRESENT_SRC_KHR;
1133 imageBarrierToPresent.image = swapChainImages[imageIndex];
1134 imageBarrierToPresent.subresourceRange = { VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1 };
1135
1136 VkDependencyInfo dependencyInfoToPresent{};
1137 dependencyInfoToPresent.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO;
1138 dependencyInfoToPresent.imageMemoryBarrierCount = 1;
1139 dependencyInfoToPresent.pImageMemoryBarriers = &imageBarrierToPresent;
1140 vkCmdPipelineBarrier2(commandBuffer, &dependencyInfoToPresent);
1141
1142 if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
1143     throw std::runtime_error("Failed to record command buffer!");
1144 }
1145 }
```



```
1146
1147 void HelloTriangleApplication::updateUniformBuffer(uint32_t currentImage) {
1148     static auto startTime = std::chrono::high_resolution_clock::now();
1149     auto currentTime = std::chrono::high_resolution_clock::now();
1150     float time = std::chrono::duration<float>(currentTime - startTime).count();
1151
1152     SceneUniformBufferObject ubo{};
1153     ubo.view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);
1154     ubo.eyePos = cameraPos;
1155     ubo.proj = glm::perspective(glm::radians(45.0f), swapChainExtent.width / (float)swapChainExtent.height, 0.1f, 10.0f);
1156     ubo.proj[1][1] *= -1;
1157
1158     ubo.lightPos1 = glm::vec3(2.0f, 2.0f, 2.0f);
1159
1160     // Rotate around Y-axis (was Z)
1161     float rotAngle = time * glm::radians(70.0f);
1162     glm::vec4 rotatingPos = glm::rotate(glm::mat4(1.0f), rotAngle, glm::vec3(0.0f, 1.0f, 0.0f))
1163         * glm::vec4(2.0f, 0.0f, 1.0f, 1.0f);
1164     ubo.lightPos2 = glm::vec3(rotatingPos);
1165
1166     memcpy(uniformBuffersMapped[currentImage], &ubo, sizeof(ubo));
1167 }
1168
```

```

shader.vert  X  shader.frag  Lab_Tutorial_Template.cpp
1      #version 450
2
3      // INs
4      layout(location = 0) in vec3 inPosition;
5      layout(location = 1) in vec3 inColor;
6      layout(location = 2) in vec3 inNormal;
7
8      // UBO (Scene Data)
9      layout(binding = 0) uniform SceneUBO {
10         mat4 view;
11         mat4 proj;
12         vec3 lightPos1; // Static White
13         vec3 lightPos2; // Rotating Red
14         vec3 eyePos;
15     } ubo;
16
17     // Push Constants (Per-Object Data)
18     layout(push_constant) uniform PushConstants {
19         mat4 model;
20         vec4 ambientMat; // Using vec4 for alignment
21         vec4 specularMat; // Using vec4 for alignment
22         float shininess;
23     } pc;
24
25     // OUTs
26     layout(location = 0) out vec3 fragColor; // Will use as Diffuse Material
27     layout(location = 1) out vec3 fragWorldPos;
28     layout(location = 2) out vec3 fragWorldNormal;
29     layout(location = 3) out vec3 fragAmbientMat; // Pass material from PC
30     layout(location = 4) out vec3 fragSpecularMat; // Pass material from PC
31     layout(location = 5) out float fragShininess; // Pass material from PC
32
33     void main() {
34         gl_Position = ubo.proj * ubo.view * pc.model * vec4(inPosition, 1.0);
35         fragWorldPos = (pc.model * vec4(inPosition, 1.0)).xyz;
36         fragWorldNormal = mat3(transpose(inverse(pc.model))) * inNormal;
37
38         // Pass vertex color as diffuse material
39         fragColor = inColor;
40
41         // Pass push-constant materials to frag shader
42         fragAmbientMat = pc.ambientMat.xyz;
43         fragSpecularMat = pc.specularMat.xyz;
44         fragShininess = pc.shininess;
45     }

```

```

shader.vert  shader.frag  Lab_Tutorial_Template.cpp
1      #version 450
2
3      layout(location = 0) in vec3 vWorldPos;
4      layout(location = 1) in vec3 vNormal;
5
6      layout(location = 0) out vec4 outColor;
7
8      layout(set = 0, binding = 0) uniform SceneUBO {
9          mat4 view;
10         mat4 proj;
11         vec3 lightPos1; float _pad0;
12         vec3 lightPos2; float _pad1;
13         vec3 eyePos;    float _pad2;
14     } UBO;
15
16     layout(push_constant) uniform ObjectPush {
17         mat4 model;
18         vec4 ambientMat;
19         vec4 specularMat;
20         float shininess;
21         float _pcPad0; float _pcPad1; float _pcPad2;
22     } PC;
23
24     void main() {
25         vec3 N = normalize(vNormal);
26         vec3 V = normalize(UBO.eyePos - vWorldPos);
27
28         // Base albedo: white (ignores per-vertex color)
29         vec3 baseDiffuse = vec3(1.0);
30
31         // Light colors
32         vec3 light1Color = vec3(1.0, 1.0, 1.0); // white
33         vec3 light2Color = vec3(1.0, 0.1, 0.1); // red
34
35         // Light 1
36         vec3 L1 = normalize(UBO.lightPos1 - vWorldPos);
37         float diff1 = max(dot(N, L1), 0.0);
38         vec3 H1 = normalize(L1 + V);
39         float spec1 = pow(max(dot(N, H1), 0.0), PC.shininess);
40
41         // Light 2
42         vec3 L2 = normalize(UBO.lightPos2 - vWorldPos);
43         float diff2 = max(dot(N, L2), 0.0);
44         vec3 H2 = normalize(L2 + V);
45         float spec2 = pow(max(dot(N, H2), 0.0), PC.shininess);
46
47         vec3 ambient = PC.ambientMat.rgb;
48         vec3 diffuse = baseDiffuse * (diff1 * light1Color + diff2 * light2Color);
49         vec3 specular = PC.specularMat.rgb * (spec1 * light1Color + spec2 * light2Color);

```

```

1080
1081 VkBuffer vertexBuffers[] = { vertexBuffer };
1082 VkDeviceSize offsets[] = { 0 };
1083 vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);
1084 vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0, VK_INDEX_TYPE_UINT16);
1085 vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1, &descriptorSets[currentFrame], 0, nullptr);
1086
1087 // Materials and draw loop unchanged...
1088 ObjectPushConstants mat1{};
1089 mat1.ambientMat = glm::vec4(0.1f, 0.0f, 0.0f, 1.0f);
1090 mat1.specularMat = glm::vec4(0.8f, 0.6f, 0.6f, 1.0f);
1091 mat1.shininess = 32.0f;
1092
1093 ObjectPushConstants mat2{};
1094 mat2.ambientMat = glm::vec4(0.0f, 0.1f, 0.05f, 1.0f);
1095 mat2.specularMat = glm::vec4(0.1f, 0.1f, 0.1f, 1.0f);
1096 mat2.shininess = 10.0f;
1097
1098 ObjectPushConstants mat3{};
1099 mat3.ambientMat = glm::vec4(0.02f, 0.02f, 0.1f, 1.0f);
1100 mat3.specularMat = glm::vec4(0.8f, 0.8f, 0.9f, 1.0f);
1101 mat3.shininess = 128.0f;
1102
1103 std::array<ObjectPushConstants, 3> materials = { mat1, mat2, mat3 };
1104
1105 for (int i = 0; i < 3; ++i) {
1106     ObjectPushConstants pc = materials[i];
1107
1108     glm::mat4 model = glm::mat4(1.0f);
1109     model = glm::translate(model, glm::vec3(-2.5f + (i * 2.5f), 0.0f, 0.0f));
1110     model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
1111     pc.model = model;
1112
1113     vkCmdPushConstants(
1114         commandBuffer,
1115         pipelineLayout,
1116         VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT, // make visible to both stages
1117         0,
1118         sizeof(ObjectPushConstants),
1119         &pc);
1120
1121     vkCmdDrawIndexed(commandBuffer, static_cast<uint16_t>(indices.size()), 1, 0, 0, 0);
1122 }
1123

```